

Pre Proyecto 2

Actividad formativa: Sistema de Colonia de Hormigas

El propósito de esta actividad es aplicar una metodología de desarrollo modular a la implementación de una **metaheurística**, de manera que lo aprendido pueda posteriormente utilizarse en problemas de optimización más complejos.

El problema de entrenamiento será el **Problema del Vendedor Viajero (TSP)** y se utilizará el **Sistema de Colonia de Hormigas (Ant Colony System, ACS)** para buscar una buena solución.

La actividad pone énfasis en **comprender, modularizar, probar y analizar** la solución.

1. Problema

Dado un conjunto de ciudades y las distancias entre ellas, se desea encontrar una ruta que:

- visite cada ciudad exactamente una vez;
- regrese a la ciudad de origen;
- minimice la distancia total recorrida.

El programa deberá utilizar ACS para construir y mejorar las soluciones.

El objetivo es encontrar la solución óptima al problema de **Berlin52**, e implementar correctamente la metodología y analizar el comportamiento del algoritmo.

2. Forma de ejecución

El programa deberá ejecutarse desde la línea de comandos indicando los parámetros necesarios. Por ejemplo:

```
python acs.py instancia.txt 50 200 1.0 2.0 0.9 12345 resultados.csv
```

Los parámetros deberán permitir, como mínimo, indicar:

- archivo de entrada;
- número de hormigas;
- número de iteraciones;
- parámetros propios de ACS (α , β , q_0);
- semilla aleatoria;
- archivo de salida.

Salida

Los resultados deberán almacenarse en un archivo de salida estructurado, por ejemplo CSV, para que posteriormente puedan ser procesados por otras herramientas.

El archivo deberá contener información suficiente para analizar los experimentos, como:

- instancia;
- semilla;
- parámetros utilizados;
- iteración;
- mejor solución encontrada;
- costo de la mejor solución;
- tiempo de ejecución.

La generación de gráficos no forma parte obligatoria del programa principal. Puede desarrollarse mediante un programa o script independiente.

3. Organización del programa

La solución deberá aplicar una separación sencilla de responsabilidades.

Como mínimo, se deberán utilizar **dos archivos principales**:

```
acs.py
operadores.py
```

acs.py

Este archivo deberá contener la **función principal del algoritmo ACS**.

La lectura del programa debe permitir reconocer con facilidad las etapas principales del algoritmo:

```
inicializar
    ↓
construir soluciones
    ↓
evaluar soluciones
    ↓
actualizar feromonas
    ↓
actualizar mejor solución
    ↓
repetir
```

El archivo **acs.py** debe utilizar las funciones implementadas en **operadores.py**.

operadores.py

Este archivo funcionará como una **biblioteca de operadores** utilizados por ACS.

Aquí deberán implementarse las operaciones necesarias para que la función principal pueda ejecutar el algoritmo, por ejemplo:

- lectura del problema;
- cálculo de distancias;
- construcción de una solución;
- selección de una ciudad;
- evaluación de una ruta;
- actualización de feromonas;
- otras operaciones que sean necesarias.

Los nombres y cantidad de funciones quedan a criterio del equipo de desarrollo, siempre que cada función tenga un único propósito.

Otros archivos

Se pueden incorporar archivos adicionales cuando aporten una función específica al trabajo, por ejemplo:

```
tests.py
graficar.py
experimentos.py
```

Estos archivos pueden utilizarse para automatizar pruebas, ejecutar conjuntos de experimentos o generar gráficos a partir de los resultados.

No es necesario incorporarlos si no son necesarios.

4. Principio de modularización

La modularización debe utilizarse para **simplificar y facilitar la comprensión del programa**.

La función principal de `acs.py` no debería contener todos los detalles de implementación.

Por ejemplo, debería ser posible leer una estructura conceptual como:

```
for iteracion in range(iteraciones):

    soluciones = construir_soluciones(...)

    evaluar_soluciones(...)

    actualizar_feromonas(...)

    actualizar_mejor_solucion(...)
```

Los detalles de cada operación estarán implementados en `operadores.py`.

La idea es que:

`acs.py` explique qué hace el algoritmo y `operadores.py` explique cómo se realizan sus operaciones.

Se busca una solución simple, clara y fácil de modificar.

5. Pruebas

Antes de realizar los experimentos finales, se deberán comprobar los principales operadores del programa.

Las pruebas deberán permitir verificar, entre otros aspectos:

- que los datos se leen correctamente;
- que las soluciones generadas son válidas;
- que el costo de una ruta se calcula correctamente;
- que las feromonas se inicializan y actualizan correctamente;
- que el algoritmo mejora o mantiene la mejor solución encontrada.

6. Experimentación

Una vez implementado y probado ACS, se deberán realizar experimentos modificando sus parámetros. Para facilitar la comparación, se deberá utilizar una semilla controlada y registrar los resultados en archivos.

Se espera analizar preguntas como:

- ¿Cómo cambia el resultado al aumentar el número de hormigas?
- ¿Cómo influye el número de iteraciones?
- ¿Qué efecto tienen los parámetros de ACS?
- ¿Cómo cambia el comportamiento utilizando diferentes semillas?
- ¿Existe una relación entre calidad de la solución y tiempo de ejecución?

Los resultados deberán poder analizarse posteriormente mediante una herramienta de procesamiento o visualización.

Por ejemplo:

```
python graficar.py resultados.csv
```

7. Transferencia a un problema más complejo

Una vez finalizado el trabajo, se deberá analizar cómo la solución desarrollada podría utilizarse como base para otro problema de optimización. Por ejemplo, al **Permutation Flow Shop Scheduling Problem (PFSP)**.

Explique brevemente:

1. ¿Qué partes de `acs.py` podrían mantenerse?
2. ¿Qué operaciones de `operadores.py` deberían modificarse o reemplazarse?
3. ¿Qué nueva información necesitaría representar una solución?
4. ¿Qué cambiaría en la función que evalúa una solución?

El objetivo es identificar qué elementos de la metodología pueden reutilizarse y cuáles dependen del problema.

8. Estructura sugerida

Una estructura simple podría ser:

```
trabajo_programacion_2/  
|  
├─ acs.py  
├─ operadores.py  
├─ tests.py  
├─ graficar.py  
|  
├─ datos/  
|   └─ instancia.txt  
|  
├─ resultados/  
|   └─ resultados.csv  
|  
├─ graficos/  
|   └─ grafico1.pdf  
|  
└─ README.md
```

No es obligatorio utilizar exactamente esta estructura.

La organización deberá ser suficientemente clara para identificar el programa principal, la biblioteca de operadores, los datos, las pruebas, los gráficos y los resultados.

9. Entrega

La entrega deberá contener:

- código fuente;
- archivos de entrada utilizados;
- archivos de resultados;
- casos de pruebas;
- programa de generación de gráficos, si corresponde;
- `README.md` con instrucciones de ejecución;

10. Consideraciones

Al finalizar la actividad, el equipo de desarrollo debería haber practicado un proceso que pueda reutilizar en otros problemas. El TSP y ACS son el contexto utilizado para practicar este proceso.

La idea es que, frente a un problema más complejo, el estudiante pueda reconocer qué parte corresponde al **algoritmo general** y qué parte corresponde a los **operadores específicos del problema**.

Se recomienda aplicar el principio **KISS (Keep It Simple)**: utilizar la solución más sencilla que permita cumplir correctamente el objetivo. Una solución clara, modular y fácil de entender es preferible a una solución innecesariamente compleja.

No se evaluará positivamente una arquitectura innecesariamente compleja (Hiperingeniería).

Hiperingeniería es un antipatrón en el desarrollo de software y la ingeniería de sistemas que consiste en diseñar o implementar una solución que excede sustancialmente los requisitos reales, actuales y funcionales de un problema. Se caracteriza por la introducción deliberada de abstracciones innecesarias, arquitecturas redundantes, flexibilidad hipotética ("por si acaso") y una complejidad técnica desproporcionada.

Esto resulta en un incremento ineficiente de los costos de desarrollo, mantenimiento y depuración, sin aportar un valor real al usuario final.

Es el arte de crear soluciones asombrosamente complejas para problemas que eran asombrosamente simples.